A GraphRAG-Based Assistant for the EU AI Act: Design, Implementation and Evaluation

Rubén Gamarro Nieto

Abstract

Large language models are increasingly used to find and summarise complex information, but their responses are not always supported by reliable sources. This limitation is particularly relevant in domains where answers should be traceable and verifiable, such as legal or regulatory information. Retrieval-Augmented Generation (RAG) can provide models with external evidence, while conventional text retrieval may fail to recover relevant information connected through explicit references between documents. This master's project designs, implements and evaluates a local GraphRAG assistant for the official English version of the EU Artificial Intelligence Act. The system extracts articles, annexes and recitals from the official document, divides them into passages for retrieval, and builds an RDF graph from the explicit references between them. The GraphRAG approach starts with BM25 text search and expands the retrieved evidence by following these references in the graph. Both Text RAG and GraphRAG use the same local Qwen2.5-1.5B-Instruct model, prompt and evidence limit, allowing a direct comparison between the two retrieval approaches. On 32 answerable test questions, coverage of the required source clauses increases from 94.8% with Text RAG to 97.9% with GraphRAG. Coverage of expected words and concepts in the generated answers also increases from 86.7% to 88.3%. The main improvements appear in questions requiring navigation through explicit statutory references. However, the observed differences do not establish a general end-to-end superiority of GraphRAG. Instead, they suggest that graph-based expansion can help recover connected legal evidence in specific cases, while limitations in answer generation and reliability still remain.

Index Terms

Graph RAG, retrieval-augmented generation, RDF, SPARQL, traceability, knowledge graph, EU AI Act.

I. Introduction

The EU Artificial Intelligence Act (EU AI Act) is a European Union regulation that establishes a common legal framework for the development, use and regulation of artificial intelligence systems across the EU, with different rules and obligations depending on how these systems are used and the risks they may pose. It is a long and complex regulation whose provisions are often connected through explicit references: one article may describe a requirement while another defines who it applies to, and an annex may provide the relevant categories of AI systems. As a result, answering a specific question may require the reader to move between several parts of the publication. Finding a paragraph that contains the words used in the question can be a useful starting point, but it may still leave relevant information undiscovered.

This project studies how a question-answering assistant can help with that task. The aim is to let a user ask a question in ordinary English, receive a short explanation and inspect the official passages behind it. The central problem is how to find enough relevant evidence before asking a language model to write the answer. A related problem is how to make the model's mistakes visible when the evidence is incomplete or is used incorrectly.

Consider a question about Article 86 of the original English publication: which area of Annex III is excluded from the individual explanation right, and what is an example of the infrastructure it covers? Reading Article 86 identifies the excluded annex point, but describing that area requires reading the annex as well. The question does not need to contain the words used to describe the infrastructure. A search method that relies mainly on matching words can therefore find the starting article while missing the passage that completes the answer. This example, included as question T22 in the evaluation, will be used throughout the report to explain the system's decisions.

Master's Programme in Big Data, Artificial Intelligence and Data Engineering, University of Malaga. September 2026.

Language models offer a convenient way to turn source material into a readable answer. However, they can also produce plausible statements without reliable evidence, as their training does not ensure that they will recall the correct version of a regulation or preserve every condition in a legal provision. For this reason, the assistant does not rely only on what the model may remember. It first searches a saved copy of the official text and then supplies selected passages with the question. This approach is known as Retrieval-Augmented Generation, or RAG.

The project adds a further step to this process. It records explicit references between parts of the regulation as a graph: articles and annexes become connected records, and a reference provides a route from one record to another. The assistant starts with a conventional text search and can then follow these routes to find additional evidence. In this report, *Text RAG* means using text search alone, while *GraphRAG* means extending the same search through the reference graph. Figure 1 illustrates the difference using the running example.

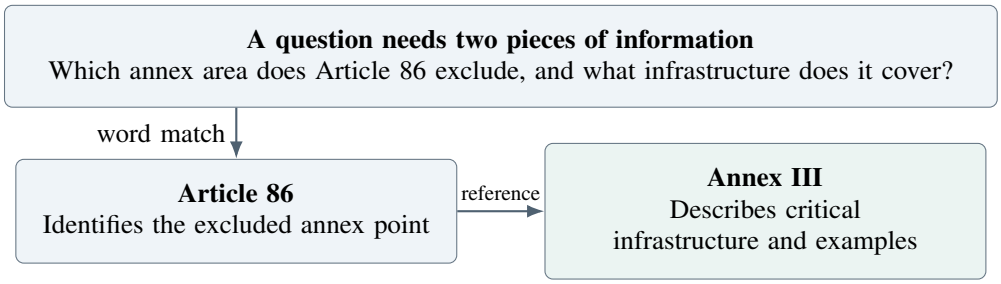


Fig. 1. Article 86 gives a starting point. The annex provides the description needed to complete the example question. The labels summarise the source rather than quote it.

The resulting application, AI Act Evidence Assistant, brings this process together in a local system that prepares the source text, builds a search index and a reference graph, retrieves relevant passages and uses a local language model to generate an answer. Its browser interface displays the retrieved passages, links to the official publication and information about how they were selected. Users can also inspect the retrieval results without generating an answer, making it possible to examine what the system found before considering the model's response.

The work has three connected objectives. The first is to build a complete assistant using an official source and a local language model. The second is to make the process easy to inspect, so that each retrieved passage can be traced back to its location in the source and each graph route can be linked to an explicit reference. The third is to evaluate whether following these references provides additional relevant evidence and whether this leads to better answers.

To make the comparison meaningful, Text RAG and GraphRAG use the same source collection, language model, answer instructions and maximum amount of evidence. A third condition, LLM-only, generates answers without receiving any source passages. The evaluation considers two main questions: whether the system retrieves the required clauses and whether the generated answer contains the expected information. Citation checks and an additional language model are also used to provide further, although limited, evidence about whether the answers are supported by the retrieved passages. Individual examples are also examined, since aggregate scores can hide important errors.

The scope is deliberately limited to the original English Official Journal publication of Regulation (EU) 2024/1689 [1]. The assistant does not track amendments, include the full text of external legislation or determine how the law applies to a particular organisation. It is an academic demonstrator for finding and inspecting information in a fixed publication. Keeping that edition fixed also makes the experiment easier to reproduce: every recorded answer can be checked against the same source material.

The main contribution is a working and inspectable application, together with evidence of where graph expansion helps. The results show a small improvement in source recovery for questions involving explicit references, while also showing that some answers remain incorrect even when the required evidence is successfully retrieved. This distinction between finding the right evidence and using it correctly is a central idea throughout the report.

The report follows the application's workflow. Section II introduces the main concepts, Section III explains how the source material is prepared, and Section IV follows a question through the assistant. Sections V and VI describe the evaluation process and discuss the results using representative examples. Section VII draws the conclusions. Finally, the appendices provide additional details about the scoring process and the steps required to reproduce the experiments.

II. Background and Related Work

The assistant combines several tools with different roles: text search finds passages that may be relevant to a question, the graph provides routes to other connected passages and the language model uses the selected evidence to generate an answer. At this stage, understanding the role of each component is more important than knowing the specific software packages used to implement them. Therefore, this section introduces these three main ideas and explains why they are combined in this project.

A. Retrieval-Augmented Generation

A language model generates text based on patterns learned during training and the information provided in its current input. However, it can produce a fluent and convincing answer even when that input does not contain enough evidence, which means that fluency alone cannot be used as a measure of correctness. When answering questions about a regulation, it is also important for the user to know which parts of the official source support the answer.

RAG adds source retrieval before generation. The system searches a collection of documents, selects passages and includes them in the model's input. That input is usually called a *prompt*. In this project, the prompt contains the user's question, instructions and the retrieved passages, allowing the model to use information from a known source instead of relying only on what it learned during training. Lewis et al. [2] introduced RAG as an approach that combines a language model with retrieved information. This project follows the general idea of retrieving relevant passages and providing them to a language model, without reproducing the specific training method proposed in that work.

In the assistant, source passages are taken from the processed publication and assigned identifiers, which allow the generated answer to refer back to the passages included in its prompt and the application to display them directly to the user. The model's weights are not modified, and the assistant is not trained on the benchmark questions. Instead, the source information is provided when each question is asked.

This design makes it easier to distinguish between two types of failure. A retrieval failure occurs when the model does not receive a passage that is needed to answer the question, while a generation failure occurs when the relevant passage is available but the model ignores it, misunderstands it or combines it with an unsupported statement. Improving retrieval can help with the first problem, but it does not automatically solve the second, and this distinction is an important part of the evaluation.

B. Text and Graph-Based Retrieval

The starting point for retrieval is BM25, an established method for ranking text by relevance [3]. In simple terms, BM25 gives a passage a higher score when it contains useful words from the question, while also considering how common those words are across the collection and the length of the passage. Repeating the same word many times does not continue to increase its importance at the same rate, making BM25 a stronger baseline than simply counting matching words.

BM25 is suitable for this project because it can search a small local collection without requiring another neural model, and its scores can also be inspected. However, it still depends on the wording of both the question and the retrieved passages. If a question refers to an article but part of the required answer appears in an annex using different words, the relevant annex passage may receive a score that is too low to be selected. A dense retriever, which compares learned representations of meaning rather than relying mainly on lexical matches, is another possible approach, but it is outside the implemented comparison and remains a useful baseline for future work.

A graph provides another way to reach this missing information by representing objects as nodes and the connections between them as edges. In this application, for example, an edge can record that Article 86 refers to

Annex III. Since this connection comes directly from the official publication, following it allows the assistant to consider passages from the annex even when their words do not closely match those used in the question.

The graph is stored using RDF, a standard way to represent simple statements about connected objects [4]. For example, the statement "Article 86 mentions Annex III" has a source, a relationship and a destination. SPARQL is the language used to ask for these connections [5]. Together, these standards provide a practical way to represent and query the reference graph, but they do not interpret the legal meaning of the references.

GraphRAG therefore uses the graph to extend BM25. It still begins with text ranking and returns a small set of source passages, but a graph connection can increase the score of a passage from a referenced part of the regulation and make it more likely to be included in the model's input. It does not directly write an answer or prove that a legal condition applies. In the running example, the graph can help reach Annex III, but the answer model must still identify the relevant excluded point and distinguish it from the other contents of the annex.

C. Related Work

Graph-based retrieval is a broad idea, and different systems use graphs for different purposes. GeAR combines graph expansion with an agent that can perform several retrieval steps [6]. Its relevance here is the use of connections to extend an initial search. The present application uses a simpler, fixed procedure: it follows a limited number of explicit references and does not use an agent to choose a new sequence of actions.

The GraphRAG approach described by Edge et al. [7] focuses on broad questions by organising extracted information into graph communities and summaries. The present project has a narrower task: finding specific clauses inside one regulation, so its graph represents explicit references that already exist in the official source. It does not introduce model-generated legal facts or community summaries. Therefore, the shared term GraphRAG refers to the general use of graph structure to improve retrieval rather than to an identical implementation or a direct reproduction of published results.

Earlier work also motivates evaluating answers and citations separately. ALCE studies language-model answers with citations [8], while RAGAS develops evaluation methods for RAG systems [9]. This project follows the general idea that an answer may appear complete even when its supporting evidence is weak, and therefore uses separate measures for source-clause retrieval and answer content, together with an automatic support check. Its reported values should not be compared directly with scores from those different benchmarks.

The support check uses Natural Language Inference, abbreviated as NLI. An NLI model compares a statement with a passage and estimates whether the passage supports the statement. Here, a separate pretrained NLI model performs this check after the answer has been generated [10]. Since the model was trained on general-domain text rather than specialised legal material, it may incorrectly assess some legal statements, so its output is treated as an additional signal for inspection rather than a final judgement of correctness.

Together, these ideas suggest a clear design. Preserve the official text, search it using a strong and understandable method, follow references where they may add evidence, and show enough of the process for mistakes to be examined. The next section explains how the source collection was prepared to support that design.

III. Source Processing and Representation

Before the assistant can answer a question, it needs a reliable collection of source passages and a record of the references that connect them. This preparation is important because every later stage depends on the material produced here. If a passage loses an introductory condition or a reference points to the wrong article, a fluent answer may hide the original processing error. The preparation process therefore preserves both readable text and the information needed to trace each passage back to its source.

A. Corpus Preparation

The source used in this project is the original English HTML publication of the EU AI Act, downloaded from EUR-Lex [1]. The project stores this file together with its source address, download date and SHA-256 hash, which

acts as a digital fingerprint that can later be used to verify that the saved file has not changed. The download date and processing date are recorded separately.

Using a saved publication gives the experiment a fixed starting point. A later download from the same address might differ, and a current consolidated edition could contain material outside the study. The assistant therefore describes its edition in the interface and keeps external legislation outside the searchable collection. This boundary is important when interpreting both an answer and a refusal: some questions cannot be answered from the selected source, even if an answer exists elsewhere.

The parser, which is the program that reads the document's structure, identifies 113 articles, 13 annexes and 180 recitals. Recitals are the introductory statements that explain the background to the regulation. Together, these parts form 306 document records, each containing an identifier, a title, its text and an official link. The expected counts are checked so that an unexpected document structure causes preparation to stop rather than silently produce an incomplete collection.

Searching whole articles would often supply more text than the model needs, especially because some articles contain several different duties or conditions. The parser therefore identifies smaller structural units, such as numbered paragraphs, definition entries and annex points, before creating retrieval passages. The processed collection contains 1050 such units and 1149 passages. Figure 2 shows how these levels are connected.

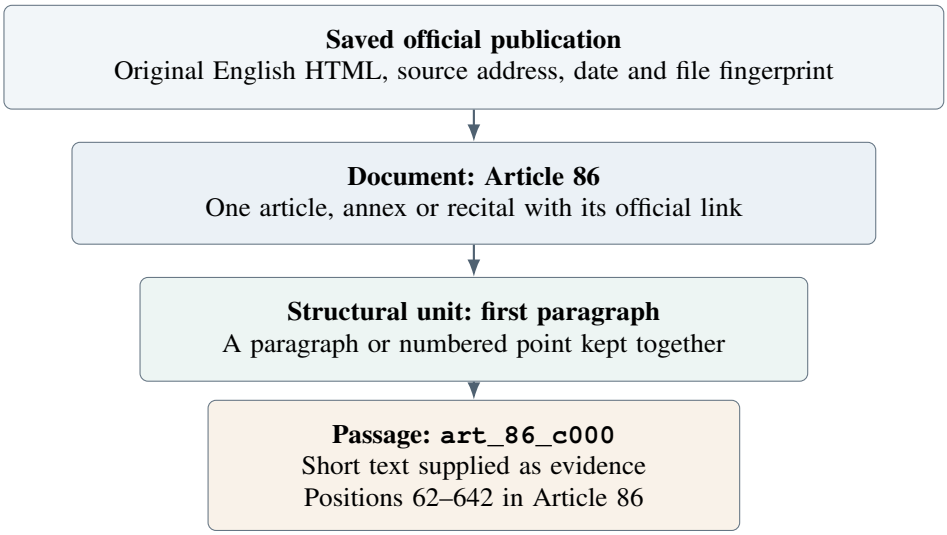


Fig. 2. How the source becomes searchable. A passage remains linked to its structural unit, its parent document and the saved official publication. The Article 86 identifiers illustrate actual records in the corpus.

Long structural units are divided into passages of at most 170 words. The splitting process prefers sentence or clause boundaries and keeps a short overlap when a unit requires more than one passage, reducing the risk of losing important context at a boundary. It also avoids combining the end of one paragraph with the beginning of an unrelated paragraph simply to fill a fixed-size window.

Some lists depend on an introductory sentence because an individual list item may describe an action without repeating who must perform it. When possible, a short introduction ending with a colon is therefore attached to the passage as a labelled lead-in. This additional source text also counts towards the evidence limit used during retrieval, preserving the context needed to understand the passage while keeping the introduction separate from the passage itself.

Every passage stores its start and end positions within the processed parent document. For example, `art_86_c000` belongs to the first paragraph record of Article 86 and covers character positions 62 to 642, with the final position excluded. A verification check reconstructs the passage by taking that exact slice of the document. The model's evidence can consequently be traced to the same text used during preparation.

The processed text is not a complete reproduction of the web page. Repeated whitespace is simplified, and numeric superscripts are written explicitly as powers: the compute threshold in Article 51 becomes `10^25`. Tables,

footnote relationships and all visual features of the original HTML are not fully represented in plain text. For this reason, the official link remains available for a reader who needs to inspect the source presentation.

B. Reference Graph Construction

Once the text is organised, the next step is to find references between its parts. A phrase such as "Article 22(1)" provides both a destination article and, when it can be resolved, a destination paragraph. The extractor records the phrase, the document containing it and its position in that document. It then checks whether the destination belongs to the indexed publication.

This distinction matters because the AI Act also refers to external legislation, and the presence of an article number alone is not enough to conclude that it refers to another article of the AI Act. The resolver therefore considers explicit external attribution, named treaties, coordinated lists and amendment contexts. External occurrences remain in the audit records but are excluded from the internal retrieval graph. The released corpus records 538 internal occurrences and 124 external occurrences.

Several occurrences can connect the same pair of documents, so the collection contains 374 unique directed document connections, fewer than the total number of internal reference occurrences. These connections preserve the source-to-destination direction because the fact that one article refers to another does not imply that the second refers back to the first. RDF also stores the documents, units, passages and individual occurrences, giving a total of 10554 statements in the exported graph.

Where a paragraph or annex point can be resolved, that more precise destination helps the retriever focus on the relevant part of a long document. For example, the graph includes references from Article 23 to Article 22(1), from Article 27 to Annex III point 5, and from Article 73 to definition 49 in Article 3. When a reference can only be resolved at document level, the system considers passages from that document rather than assigning a more precise destination that is not supported by the recorded reference.

The running example illustrates this limit. The reference used for T22 connects Article 86 to Annex III only at document level, so the retriever must then select candidate passages within the annex. Although the required infrastructure point is successfully retrieved, the graph edge does not encode the meaning of the exclusion or directly identify point 2. Keeping this distinction clear prevents a graph route from being interpreted as proof that the resulting answer is correct.

Source checking is carried out at several levels. All passages are checked against their saved text slices, and accepted references are checked against their recorded phrases and graph connections. A targeted test set also covers 40 external-reference occurrences spanning 33 document pairs. These tests provide evidence for specific cases, but they do not establish the precision or recall of reference extraction across every possible reference expression in the regulation.

At the end of this preparation process, the assistant has a collection of short source passages and a graph that connects them through their documents and structural units. These records are saved independently of the language model, allowing retrieval to be inspected and tested before any answer is generated. The next section follows what happens when a user asks a question using this prepared collection.

IV. Question Processing and Answer Generation

The prepared corpus provides the information needed to answer questions, but the system must still transform a user's question into a short answer without losing the connection to the official source. The application handles this process in three main stages: evidence selection, answer generation and presentation for inspection. Figure 3 shows the complete workflow. The graph affects evidence selection, while the language model and interface are shared by both RAG modes.

A. Evidence Retrieval

When a question arrives, the application first checks that it is not empty or too long and that the selected mode is recognised. In both retrieval modes, the system then searches the prepared passages using text that includes the

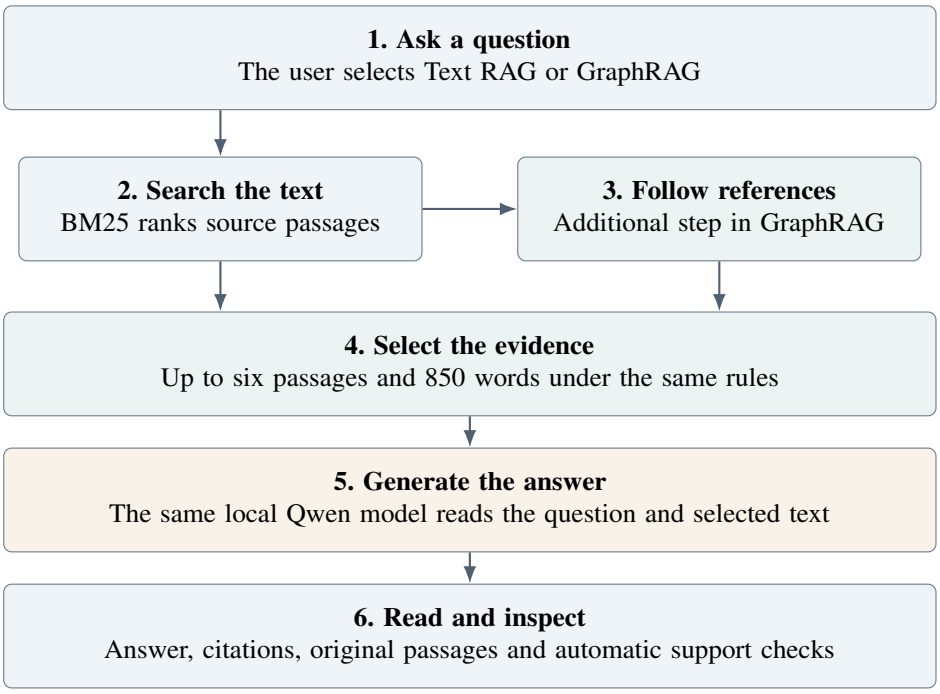


Fig. 3. The path followed by a question. Text RAG and GraphRAG use the same generator and evidence limits. The additional reference search changes which passages may be selected. LLM-only bypasses retrieval and supplies no passages.

passage itself, its title and any associated lead-in. Including this additional information helps preserve the context of short fragments such as numbered list items.

BM25 gives each passage a score based on its word match with the question. Both retrieval modes also recognise explicitly named provisions. If the user asks about Article 86, passages from that article receive a shared preference, while a matching paragraph can receive an additional preference. These rules are applied before graph expansion and are identical in Text RAG and GraphRAG, meaning that the graph is compared against a text baseline that already makes use of explicit article references.

Text RAG selects passages from this ranking. GraphRAG takes the same ranking and chooses up to two starting documents, called seeds. If the question explicitly names indexed provisions, the seeds are restricted to those provisions. Otherwise, the best positive text scores provide the starting points. This lets the graph begin from a plausible part of the publication instead of exploring the whole regulation indiscriminately.

From these seeds, the retriever queries the RDF graph for referenced documents using SPARQL and RDFLib [11]. The application then checks the corresponding reference occurrences and considers passages from each destination. When the reference can be resolved to a specific paragraph or annex point, candidates are taken from that structural unit. Otherwise, passages from the destination document are considered.

Passages reached through a reference can receive an additional score, called the *graph bonus*. This bonus is larger when both the starting passage and the source unit containing the reference are relevant to the question, and it decreases as the graph route becomes longer. A reference resolved to a specific paragraph or point also receives a small preference because it identifies a more precise destination. The complete scoring formula and parameter values are provided in Appendix A, where they define how strongly graph connections can modify the original text ranking.

Graph expansion is bounded. The selected configuration allows at most two reference steps, or hops, and restricts the number of candidates examined at each step. It avoids cycles within a route and keeps the strongest contribution to a candidate rather than adding every route together. This prevents a frequently mentioned document from receiving an unlimited benefit simply because it has many connections. It also means that relevant evidence can still be missed when it lies outside the allowed search.

After ranking, both modes apply the same evidence limits: a maximum of six passages, no more than three passages from the same document and a total limit of 850 words, including lead-ins. Passages that would exceed the remaining space are skipped rather than cut midway through their text. Substantially overlapping passages from the same unit are suppressed. The aim is to give both systems the same maximum opportunity to provide evidence to the model.

The T22 example illustrates the effect of graph expansion. Both modes retrieve three passages from Article 86, but GraphRAG also promotes two passages from Annex III, replacing two recitals selected by Text RAG. One of these passages contains the infrastructure information required by the question. Its score increases from approximately 0.325 to 0.636 after receiving a graph bonus of about 0.311, which is sufficient to include the relevant source text in the final evidence.

This result should still be interpreted at passage level. For T22, the graph follows a single document-level reference from Article 86 to Annex III. It does not require a second hop, and it does not encode the meaning of the excluded point. Both systems select six passages, but their actual contexts differ in length: 418 words for Text RAG and 299 for GraphRAG. Equal maximum budgets therefore control the upper limit, rather than guaranteeing identical input lengths.

B. Answer Generation and Support Checking

Once the passages have been selected, the assistant builds the model's input. Each passage appears with an identifier and its source information. The instructions ask the model to answer all parts of the question, preserve the responsible actors and relevant conditions, and cite the passages supporting its statements. Expected benchmark answers are never included, so the model must generate its response using only the question, instructions and available evidence.

The generator is Qwen2.5-1.5B-Instruct [12], which runs locally through llama.cpp [13] and contains approximately 1.5 billion parameters. The project uses the Q4_K_M GGUF quantisation together with a fixed llama.cpp build to reduce memory requirements and support local CPU execution. This is a practical implementation choice, and the study does not claim that this model is optimal for legal question answering.

The model has a context limit of 4,096 tokens. Before each request, the application uses the server tokenizer to verify the complete input together with the space reserved for generation. Requests that exceed the available context produce an explicit error rather than silently removing retrieved evidence. This ensures that the evidence recorded by the retrieval stage corresponds to the evidence actually received by the model.

Both RAG modes use the same answer instructions and generation settings. The prompt requests answers of no more than 160 words, while the server enforces a maximum of 320 output tokens and marks responses that reach this limit as truncated. LLM-only follows the same main answer requirements but receives no source passages or citation instructions. The three conditions therefore allow the evaluation to examine the contribution of source context as well as the additional effect of the graph.

After generation, the application extracts citation identifiers from the answer and verifies that they correspond to passages actually provided to the model. This detects invented or malformed citation labels, but it does not determine whether the cited passage truly supports the associated statement. A model may cite an existing passage while still misunderstanding or reversing its meaning, which is particularly important for legal provisions containing exceptions or prohibitions.

The browser also offers an automatic support check using the separate NLI model introduced earlier. The checker divides the answer into sentences and compares each cited sentence with windows of the cited text. It retains the highest support score for that sentence and the interface can then draw attention to statements that may need closer reading. The checker does not rewrite the answer, change retrieval or give the generator another attempt.

This support check has limits. It may fail when a statement needs information from several passages or when a legal condition depends on a heading outside the selected window. It can also accept a sentence supported by one citation even if the same sentence includes extra irrelevant citations. Its scores are therefore shown alongside the source text. They are not a probability that the legal answer is correct.

Failure handling is part of this process. If retrieval returns no evidence, the RAG modes produce an insufficient-evidence response without calling the generator. An unavailable model, an invalid question or a request that exceeds the context limit produces a visible error. The web application admits one generation at a time, so a second concurrent request receives a busy response. These behaviours make operational failures distinguishable from an answer that the model generated incorrectly.

C. Application Interface and Deployment

The browser is the main way to use the assistant. A user enters an English question, chooses a mode and either asks for an answer or inspects retrieval alone. GraphRAG is the default, with Text RAG and LLM-only available for comparison. The answer is shown separately from the official evidence so that readers can see the difference between generated text and source text.

Figure 4 shows an actual application response to the running example. The selected citation opens the corresponding Annex III passage in the evidence panel. This makes the result inspectable in a direct way: the user can read what the assistant said and compare it with the material it received.

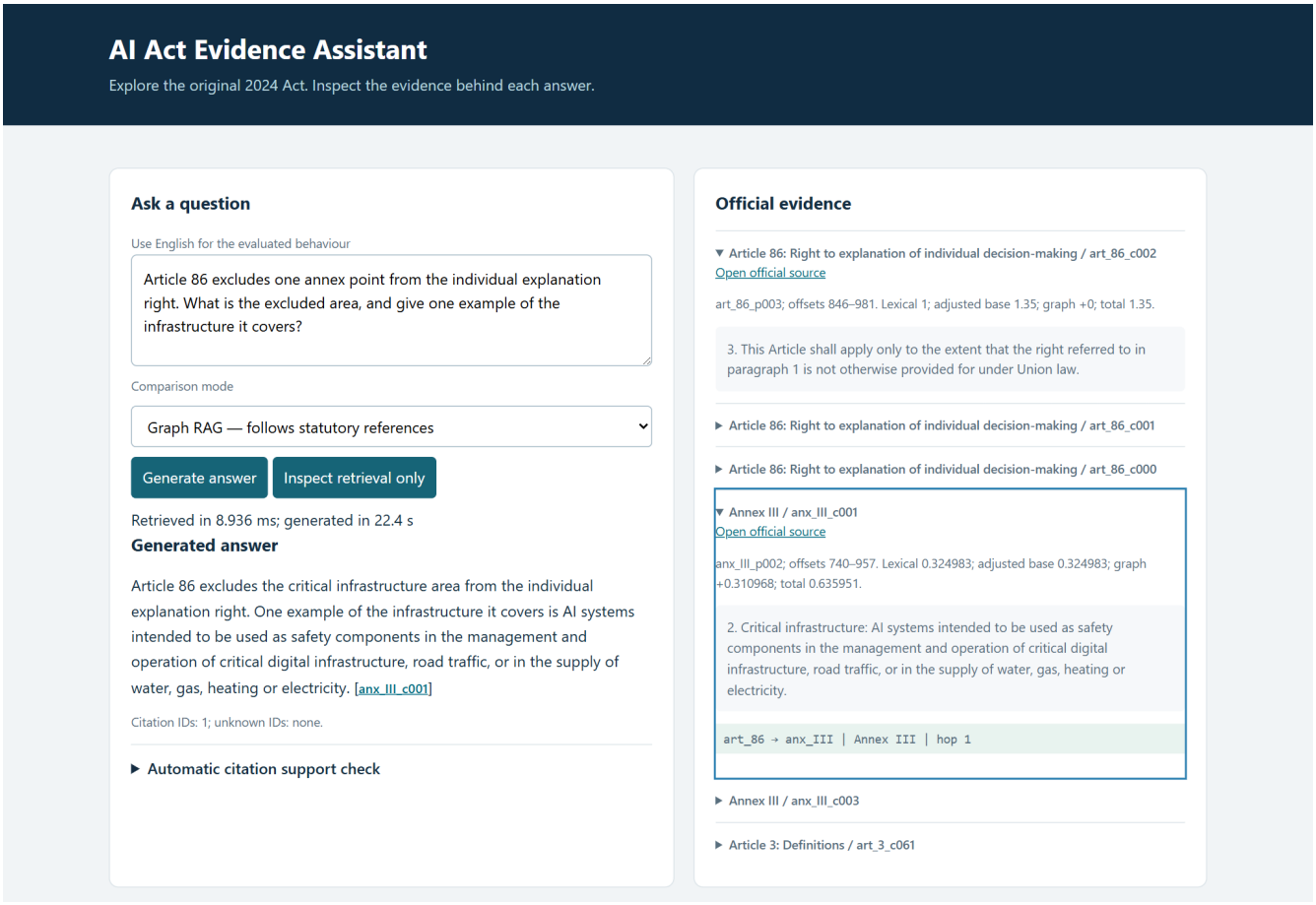


Fig. 4. Actual application screenshot. The question and generated answer appear on the left. On the right, the expanded Annex III passage shows its source text, official link and retrieval scores. The citation connects the two views.

Each evidence record contains its document and passage identifiers, source positions and link to the official publication. The interface also separates the original text score, adjusted base score, graph bonus and final score. Where graph expansion contributed to selected evidence, the interface also displays the route and the source phrase behind the connection. A passage selected only by text search does not need a graph route to justify its presence.

The interface exposes automatic support results sentence by sentence. It also provides saved evaluation summaries, which describe the recorded experiment rather than the current user's question. Asking a question does not rebuild

the benchmark or recompute those tables. This separation prevents an aggregate research result from appearing to be a confidence estimate for an individual answer.

The same Python components used by the web application also support command-line execution and automated evaluation. Source preparation, retrieval, generation and support checking are separate components. The application programming interface, or API, connects the browser to those components through HTTP requests. Search data are loaded and reused by the application process, and the language model runs as a separate service. This arrangement keeps the model service independent of the interface and allows retrieval to be used without generating text.

The local deployment uses Docker Compose to manage the application and model services. An initialisation service downloads and verifies the required model files and creates the API key before the model service starts. Once the model is healthy, the application service becomes available. Figure 5 shows the main connections.

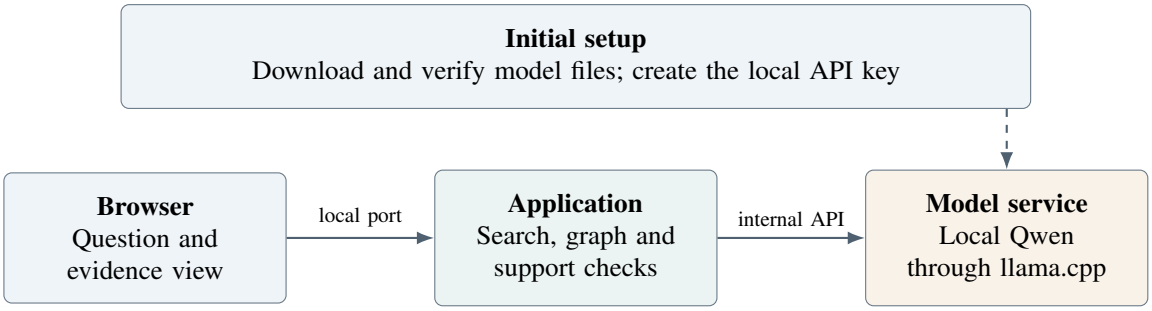


Fig. 5. Local deployment architecture. Initialisation prepares shared assets before startup. The browser reaches the application through the local web port, and model requests stay inside the Compose network. Persistent volumes retain the models, key and results.

The browser normally accesses the application at http://127.0.0.1:8000, while the model service remains internal to the Compose network. The app and model services run without root privileges, and the API key stays on the backend. The language model receives only text and has no access to files or tools for performing external actions. Generated content is displayed as text rather than executed as HTML. These are concrete implementation choices, while public hosting and multi-user operation remain outside the evaluated scope.

The completed application therefore gives the user more than a final paragraph: it exposes the evidence, the selection process and visible failure information. The next question is how to assess whether this design works. That requires measuring retrieval and answers separately, using the same questions under controlled conditions.

V. Evaluation Methodology

A working interface does not show whether an assistant finds the right evidence or writes reliable answers. The evaluation therefore follows the same sequence as the system: first inspect the retrieved passages, then assess the answer and its citations. The main comparison is between Text RAG and GraphRAG. LLM-only provides a further reference point for understanding what changes when source passages are supplied at all.

The study addresses three questions. Does following references recover clauses that BM25 misses? Does the additional evidence improve the information included in the answer? What limitations remain in citations, support, refusal behaviour and response time? These questions are assessed using the recorded experiment, with all test questions and failures retained.

A. Evaluation Dataset and Experimental Setup

The dataset contains 60 English questions about, or deliberately outside, the selected source. Twenty questions form the development set, used to choose retrieval settings. The remaining 40 form the test set used to report results. This is a developer-created diagnostic dataset: the test questions were available during system development,

and independent users or legal experts did not create or validate them. The term "test" describes their role in the experiment, rather than an unseen external benchmark.

Questions are grouped by the kind of information they require. Direct questions ask for a fact that can be found in a provision, such as the minimum number of regulatory sandboxes. Reference questions identify a starting article and require following a connection to another part of the source. Conditional questions involve roles, exceptions or deadlines. Unanswerable questions request information that is not available in the selected publication, including plausible requests for absent information rather than only unrelated topics.

The development set has six direct, six reference, four conditional and four unanswerable questions. The test set has 12 direct, 12 reference, eight conditional and eight unanswerable questions. Thus, 32 test questions have annotated source answers, and eight assess the response when the source is insufficient. Figure 6 shows how the two sets are used.

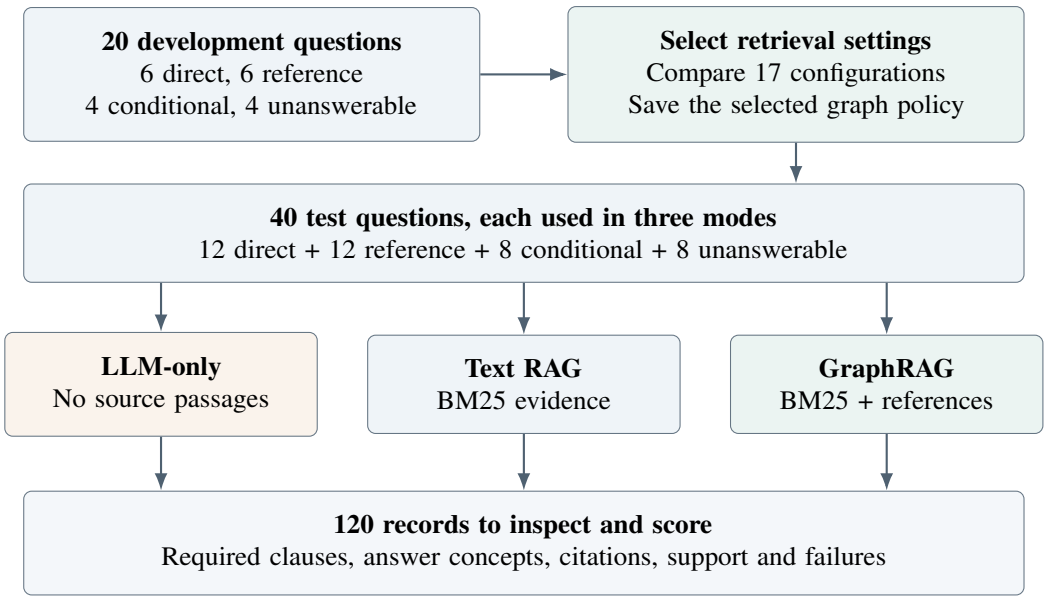


Fig. 6. Evaluation design. Development questions select the retrieval settings. Each of the 40 test questions is then evaluated in all three modes. Reference annotations are used for scoring and are never supplied to retrieval or generation.

Each answerable question has annotations that specify required source documents, complete clauses, short identifying quotes and expected answer concepts. A clause annotation points to an exact span of the processed source. These spans are checked automatically against the corpus before evaluation. The annotations provide an inspectable definition of the expected evidence, although they do not exhaust every legally equivalent way to answer a question.

Seventeen retrieval configurations are compared on the 16 answerable development questions. They vary the weight of the graph contribution, the number of reference steps and the direction of traversal. A disabled-graph configuration is included, so the procedure can select no graph contribution if that performs best. Configuration selection prioritises complete-clause coverage and then applies predefined tie-breaking rules. The complete settings and selection procedure are reported in Appendix A.

The selected configuration uses a graph weight of 0.25, up to two outgoing hops and up to two seeds. Its development clause coverage is 84.4%, compared with 75.0% for BM25 with the graph disabled. The selected settings are saved before the test retrieval is assessed, and the deployed application reads those same settings. Test results are not used to choose a different graph weight or traversal depth.

Every test question is evaluated using LLM-only, Text RAG and GraphRAG, giving 120 question–system records. Text RAG and GraphRAG use the same corpus, generator, prompt and maximum evidence budget, and neither receives expected answers, required source spans or benchmark labels. The graph is the intended retrieval difference between them. Actual evidence lengths and order can still differ, because choosing passages is precisely the part of the system being compared.

B. Evaluation Metrics

The first measure asks whether the retrieved evidence contains the clauses needed for the question. *Complete-clause coverage* is the fraction of annotated clauses fully covered by the selected passages. For a question requiring two clauses, retrieving only one gives 50%; retrieving both gives 100%. Coverage is first calculated for each question and then averaged, giving every question equal weight. A reported value of 97.9% therefore does not mean that 97.9% of questions received a correct answer.

The check uses source positions, rather than simply asking whether the correct article appears somewhere in the results. Several overlapping passages can jointly cover a clause, but a gap in the required interval prevents complete coverage. Additional measures record whether the required documents and short identifying quotes were found. These help investigate retrieval failures, while complete-clause coverage remains the main measure because a relevant article can contain many irrelevant passages.

The next measure asks whether the generated answer includes expected concepts. The implementation calls these concepts *facets*. For example, an answer may need to name an actor, a location and a form of appointment. *Answer-facet coverage* counts how many such concepts match a predefined set of word patterns. Matching is performed after citation labels are removed, and numerical rules distinguish values such as six and sixteen.

This measure is useful for checking omissions across many answers, but it is limited. An answer can contain the expected words while stating the wrong relationship between them, or it can also use a valid paraphrase that the patterns miss. Facet coverage is therefore described throughout the report as a measure of expected answer content, not as legal accuracy. The case analysis includes an answer that receives full facet coverage despite contradicting its own opening sentence.

Citation assessment begins with two simpler checks. Citation presence records whether the answer cites any recognised supplied passages and identifier validity checks whether cited identifiers belong to the actual evidence. A response without citations has no defined validity score. A valid identifier establishes that the source was available, but it cannot establish that the source supports the claim.

To assess support more directly, the separate NLI model compares generated sentences with source text. *Reference support* uses the annotated source clauses as the evidence against which a sentence is checked. This can be calculated for LLM-only as well as for the RAG modes. *Cited-sentence support* uses the passages that the answer actually cites. It measures supported sentences among those with citations. A separate *support coverage* measure includes all answer sentences, so uncited claims remain visible in its denominator.

These support measures also use per-answer scores before averaging across answers. Undefined values are excluded rather than treated as successful outcomes. In particular, answers without citations do not receive perfect cited-sentence support. The published cited-sentence averages use 29 Text RAG answers and 27 GraphRAG answers, whereas the main source and answer-content measures use all 32 answerable questions.

The NLI threshold is fixed at 0.8, and the same checker is used for both RAG conditions. For a cited sentence, the highest score from its cited passage windows is retained. This means an irrelevant extra citation is not necessarily penalised if another citation supports the sentence. The measure differs from checking the necessity of every individual citation, and it is not ALCE's citation-level precision. The model's general-domain training and windowed input also limit its ability to assess legal meaning.

Finally, the evaluation records refusals, incomplete answers and time. *Abstention* means that the model follows the specified refusal wording when the answer is absent from the source. It is a strict protocol check and may miss a useful expression of uncertainty in different words. A response that reaches the output-token limit is recorded as truncated. Generation time measures actual model calls and excludes the later NLI check, so it is not the same as the total waiting time in the browser.

C. Validation and Uncertainty Analysis

Before considering answer quality, the implementation is checked for specific software properties. The test suite contains 83 passing automated tests. They check exact source slices, reference attribution, evidence limits, deterministic ranking, citation parsing, metric behaviour and API failures. Tests also verify that disabling the graph reproduces the text-only ranking and that graph expansion can reach a passage with no matching query words.

These tests confirm specific expected behaviours of the implementation, but they do not establish the correctness of arbitrary generated answers.

Additional retrieval experiments, called ablations, help explain what contributes to the result. TF-IDF provides an alternative lexical baseline, while the graph-disabled condition checks equivalence with BM25. One-hop and two-hop conditions test whether longer traversal helps. A random graph provides a control in which document connections are rearranged while each document's numbers of incoming and outgoing connections are preserved. Precise paragraph targets are removed in that control, so it tests the combined effect of reference structure and target specificity.

The recorded generation run shuffles question order using a fixed seed and rotates the order of the three modes. Generation uses temperature zero and a fixed request seed to reduce avoidable variation, although these settings do not guarantee identical outputs across different hardware or software versions. Request-level prompt caching is disabled. When two complete model requests are exactly identical within the experiment, the later record reuses the already generated response and is explicitly marked as such.

Consequently, the 120 records contain 107 actual scored model calls and 13 reuses. Reused records remain part of the question-level comparison but do not provide additional timing observations. Two additional calls repeat a development prompt outside the scored test and produce identical outputs. This provides a limited check of repeatability for one example rather than evidence of universal determinism.

For each question, the experiment saves the retrieved passages, complete model request, generated answer and resulting scores, allowing individual results to be traced back to their inputs. However, because the test questions were available during development, the study should be interpreted as a diagnostic evaluation rather than an independent blind assessment.

To describe the size and stability of differences, the evaluation uses 5,000 paired bootstrap resamples. Each resample selects questions with replacement and recalculates the mean difference between GraphRAG and Text RAG, producing a descriptive 95% interval. Pairing matters because the two systems are answering the same questions. The intervals are descriptive: they do not correct for developer involvement in the question design or for shared source topics.

This protocol supports a careful reading of the results. It can show which clauses were recovered, what the model produced and how scores change under controlled retrieval variants. It cannot by itself establish performance for all AI Act questions or all language models. The next section combines the aggregate scores with concrete answers to make the practical meaning of the comparison clear.

VI. Results: where the graph helps and where it falls short

The recorded experiment shows that supplying source evidence makes a substantial difference to expected answer content, while adding the graph produces a smaller, more focused change. The useful question is therefore where that change comes from and what it means for the final answer. This section starts with source recovery, follows the running example through generation, and then examines citation support and the limits of the evidence.

The main scores are summarised in Table I. The source-clause and answer-facet values use 32 answerable questions. Refusal uses the separate set of eight unanswerable questions. Support values are automatic estimates, and time is the mean over actual model calls across all 40 questions, excluding reuses. These different denominators matter when comparing the columns.

TABLE I
Recorded comparison. Scores are percentages except time, which is in seconds. "Clauses" measures source recovery. "Facets" measures expected answer concepts. NLI columns estimate reference support and cited-sentence support.

System	Clauses	Facets	NLI ref.	NLI cite.	Refusal	Time (s)
LLM only	–	27.6	1.1	–	87.5	5.1
Text RAG	94.8	86.7	20.4	27.6	62.5	21.4
Graph RAG	97.9	88.3	24.1	33.3	62.5	20.5

A. Retrieval Results

Across the answerable questions, mean complete-clause coverage increases from 94.8% with Text RAG to 97.9% with GraphRAG, a difference of 3.1 percentage points. Both modes already recover all annotated clauses on the 12 direct and eight conditional questions. The additional recovery occurs in the reference group, where coverage rises from 86.1% to 94.4%.

The whole improvement comes from T18 and T22. In each case, Text RAG covers one of two required clauses and GraphRAG covers both. The remaining 30 answerable questions have equal clause coverage. Averaging the two gains across all 32 questions gives

$$\frac{0.5 + 0.5}{32} = 0.03125,$$

or 3.125 percentage points before rounding. This is a small change in the overall average, with a clear explanation in the individual source records. It is not a small improvement on every question.

Figure 7 separates the question groups. The left-hand plot shows where source recovery improves, and the right-hand plot shows whether expected answer concepts also improve. The graph's source-recovery gain is concentrated in reference questions. Direct and conditional questions leave no room for a higher clause score in this diagnostic because BM25 already reaches 100%.

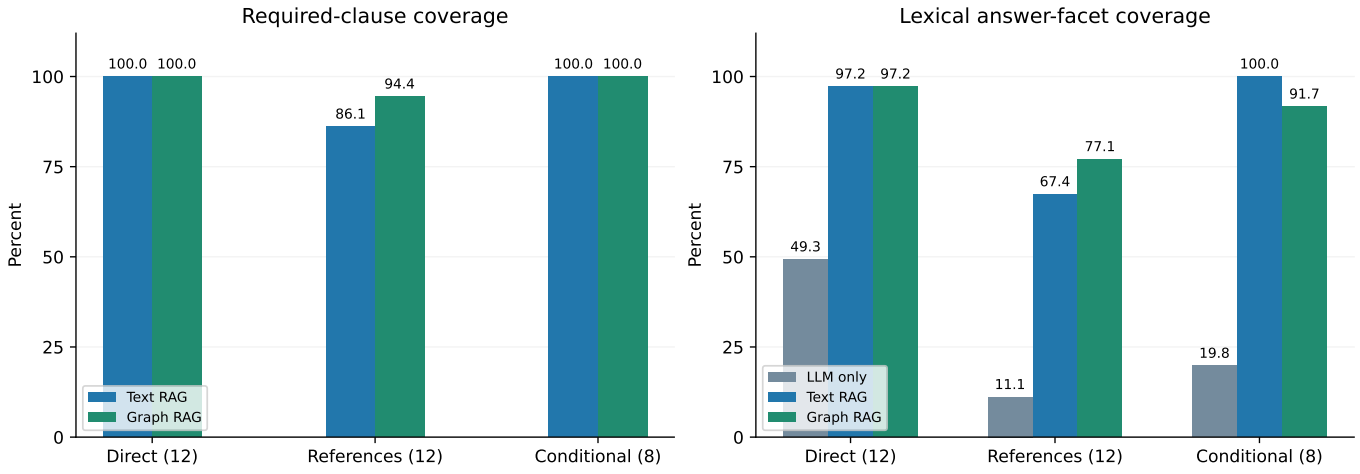


Fig. 7. Source-clause coverage and answer-facet coverage by question group. The labels give the sample sizes. The right-hand plot measures expected concepts in the answer, not expert-rated correctness.

For T22, the extra Annex III passage contains the infrastructure description needed to complete the question introduced in Section I. Text RAG retrieves the starting article but misses that required annex clause. GraphRAG follows the article's reference and promotes the annex passage into the evidence. This is the intended use of the graph: it provides a route to source content whose wording is not fully expressed in the question.

T18 also requires an infrastructure example, reached from Article 49. Both questions ultimately use the same Annex III point. The two improvements are therefore related examples rather than independent evidence across different areas of the regulation. The result supports a useful local mechanism, but a wider test would need references leading to many different destinations.

The retrieval ablations provide further context. As shown in Table II, BM25 achieves 94.8% overall complete-clause coverage compared with 83.9% for TF-IDF, indicating that the main comparison does not rely on the weaker lexical baseline. Disabling graph expansion reproduces the BM25 result exactly, confirming that the graph-disabled condition does not introduce an unintended ranking change.

Both the one-hop and selected two-hop configurations achieve 97.9% complete-clause coverage on the test set. Although the development procedure selected two hops, the test does not demonstrate an additional benefit from the second hop. The random graph also improves on BM25, reaching 96.4% overall and 90.3% on reference questions. The real graph is better on these aggregate values, but part of the improvement can arise from changing connectivity and ranking more generally. This makes the source routes and individual cases necessary to interpret the result.

TABLE II
SOURCE-CLAUSE COVERAGE UNDER RETRIEVAL VARIANTS, IN PERCENT. THE SELECTED GRAPH USES TWO OUTGOING HOPS. THE RANDOM CONTROL CHANGES DOCUMENT CONNECTIONS AND REMOVES PRECISE TARGET-UNIT INFORMATION.

Variant	All answerable (32)	Reference (12)	Direct (12)
Text search (TF-IDF)	83.9	73.6	83.3
Text RAG (BM25)	94.8	86.1	100.0
Graph disabled	94.8	86.1	100.0
Graph, one hop	97.9	94.4	100.0
Graph, two hops (selected)	97.9	94.4	100.0
Random graph	96.4	90.3	100.0

B. Answer Results and Case Analysis

Mean answer-facet coverage is 27.6% for LLM-only, 86.7% for Text RAG and 88.3% for GraphRAG. Supplying source passages is associated with much greater coverage of the expected concepts than giving the model the question alone. The additional gain from the graph is 1.6 percentage points overall. On reference questions, the score rises from 67.4% to 77.1%.

The overall change includes gains and a loss. GraphRAG improves facet coverage on three answerable questions, ties on 28 and performs worse on one. In the conditional group, the score falls from 100.0% to 91.7%, even though source-clause coverage stays at 100%. These cases demonstrate why retrieval quality and generated answer content must be evaluated separately.

T22: the added evidence leads to a more useful answer

The question asks for the annex area excluded by Article 86 and an infrastructure example. Text RAG identifies the annex point but gives a circular description of its area, referring to systems under that point. Its answer has 50% facet coverage. GraphRAG names critical infrastructure and gives examples from the recovered annex passage, reaching 100% facet coverage.

> **GraphRAG, excerpt:** "Article 86 excludes the critical infrastructure area from the individual explanation right. One example of the infrastructure it covers is AI systems intended to be used as safety components in the management and operation of critical digital infrastructure, road traffic, or in the supply of water, gas, heating or electricity. [anx_III_c001]"

This case connects the retrieval trace with the user-visible benefit. The model receives information that was missing from the Text RAG evidence and incorporates it into the answer, while the cited source remains available for inspection. Under the evaluation annotations, the GraphRAG answer therefore contains more of the expected information, although its broader legal adequacy has not been independently assessed.

T18: the evidence improves, but the answer still names the wrong annex

The question asks which annex area Article 49 assigns to national registration and requests an infrastructure example. GraphRAG retrieves both required clauses, increasing coverage from 50% to 100%. Nevertheless, its generated answer begins by assigning the area to Annex VIII. The annotated destination is point 2 of Annex III.

> **GraphRAG, excerpt:** "Article 49 assigns the Annex VIII area for national registration. One example of an infrastructure from Annex III is AI systems intended to be used as safety components in the management and operation of critical digital infrastructure, road traffic, or in the supply of water, gas, heating or electricity."

The answer includes a relevant infrastructure description, and its facet score increases from 33.3% to 66.7%. However, the wrong annex assignment remains a substantive error. Figure 8 contrasts this with T22. The same improvement in clause coverage can lead to different outcomes during generation.

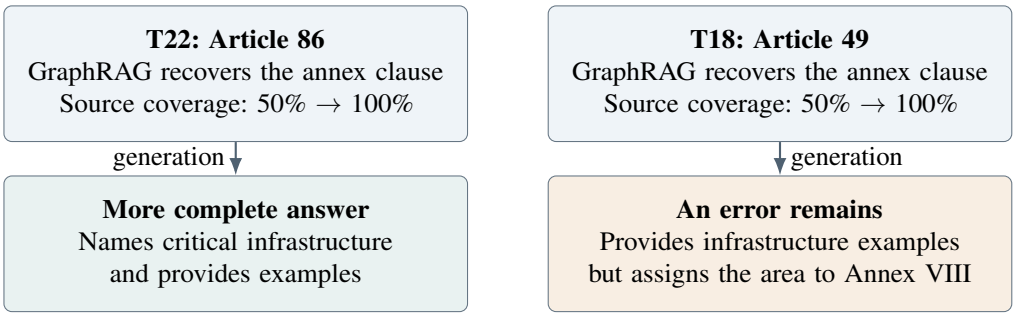


Fig. 8. Two recorded outcomes of improved retrieval. T22 uses the newly recovered information in a more complete answer. T18 still names the wrong annex despite receiving both required clauses.

T31: expected words can appear in a contradictory answer

This question asks whether an importer can place a system on the market when there is sufficient reason to believe that the system or its documentation is falsified. Both retrieval modes recover all annotated clauses, but both generated answers begin with "Yes", despite the supplied high-risk-system clause stating that the system must not be placed on the market until it has been brought into conformity.

The Text RAG answer then repeats that prohibition, contradicting its own opening. Its facet coverage is still 100% because it contains the expected concepts. GraphRAG omits the prohibition and receives 33.3%. This example demonstrates an important limitation of facet coverage: a high score indicates the presence of expected content, not that the answer expresses that content correctly.

What the other inspected cases add

T14 asks which providers must appoint a representative, where that representative must be established and how the appointment is made. Both modes have full clause and facet coverage. Text RAG cites several passages, while GraphRAG gives a more focused answer citing Article 22. This makes citation selection worth inspecting even when the headline content score is unchanged.

T15 asks for the two financial uses reached through Article 27. Both modes recover only one third of the required clauses and name the annex point numbers without explaining the requested uses, giving zero facet coverage. The graph therefore does not solve every reference question. The annotated uses concern creditworthiness and life and health insurance, which the displayed answers fail to state.

T05 asks for the minimum number of operational regulatory sandboxes each Member State must ensure by the date specified in the question. Both answers contain the expected minimum and have full clause and facet coverage, but neither supplies a recognised passage identifier. An answer can therefore meet the content rubric while falling short of the application's citation format. Taken together, these cases show why no single reported score summarises the whole user experience.

C. Citations, uncertainty and practical limits

Recognised citations appear in 90.6% of answerable Text RAG responses and 84.4% of GraphRAG responses. In counts, these are 29 and 27 of the 32 answers. The graph's higher average facet score is therefore accompanied by lower citation presence. Validity among cited identifiers is 100% in both conditions, but the examples above show why that alone is insufficient.

Automatic cited-sentence support is 27.6% for Text RAG and 33.3% for GraphRAG, averaged only over answers where that score is defined. When uncited sentences are included, support coverage is 18.2% and 20.3%. Reference support, assessed against annotated clauses, is 20.4% and 24.1%. These values suggest that supplying or citing a passage often fails to produce sentences that the automatic checker recognises as supported.

The scores should be interpreted alongside the checker's limitations. A low value can reflect an unsupported claim, but it can also reflect a mismatch between legal wording and a general-domain NLI model. A high value can

overlook an irrelevant extra citation or an omitted condition. The saved sentence-level results are therefore useful evidence for a later human review, rather than a substitute for it.

The eight unanswerable questions reveal another limitation. LLM-only follows the specified abstention protocol on seven questions, while Text RAG and GraphRAG do so on five, corresponding to 62.5% for each RAG condition. Because this is a strict wording-based measure, it does not recognise every reasonable expression of uncertainty. In TU05, both RAG systems use a useful expression of uncertainty that the strict detector misses. TU06 instead speculates about future consolidated law, and TU07 substitutes generic information for an unavailable certificate number. These last two examples reflect failures to respect the source boundary.

Among answerable questions, one response in each RAG condition reaches the token limit, corresponding to 3.1%. Mean generation times over actual calls across all test questions are 5.1, 21.4 and 20.5 seconds for LLM-only, Text RAG and GraphRAG. These values describe the measured model calls. The different reuse patterns leave different timing samples for the modes, and NLI time is excluded. The slightly lower GraphRAG mean therefore does not establish a speed advantage or represent total browser latency.

The paired bootstrap intervals further limit the strength of the observed differences. The mean clause-coverage difference is 3.1 points, with a 95% interval of [0.0, 7.8]. The overall facet difference is 1.6 points, with interval [-4.7, 7.3]. The first touches zero and the second crosses it. The reference-group intervals also include zero at their lower endpoint. Figure 9 shows the recorded intervals by group.

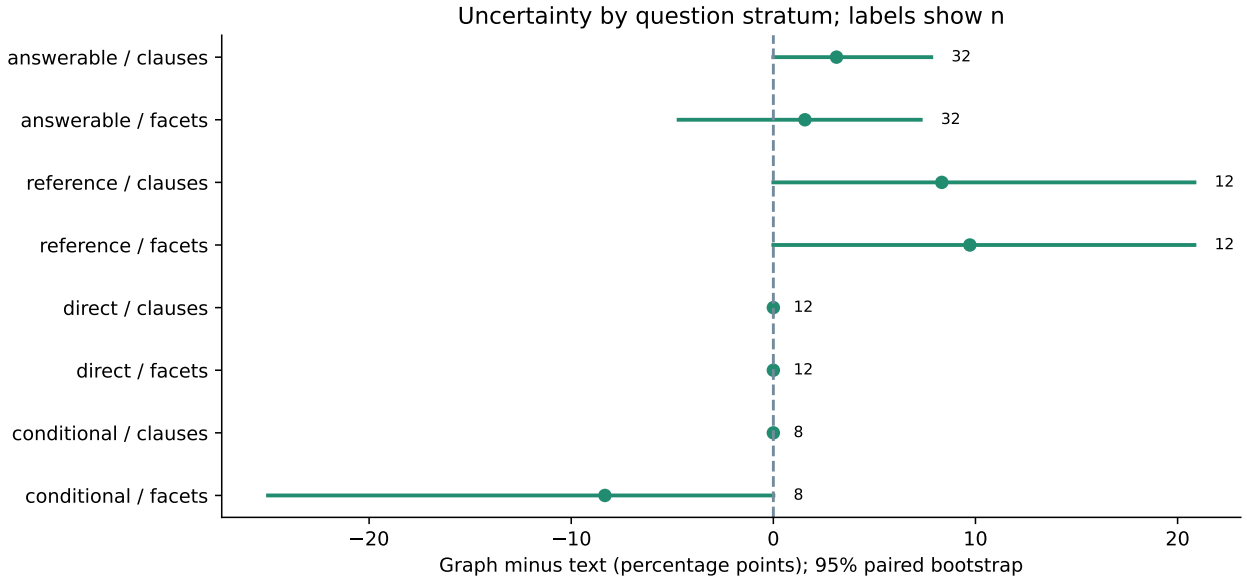


Fig. 9. Descriptive 95% intervals for GraphRAG minus Text RAG. The vertical line marks no difference. Question-level resampling does not account for shared source topics or developer selection.

The reference-support difference is about 3.6 percentage points, with interval [0.0, 8.3] over 32 paired answers. Like the other estimates, it describes the recorded diagnostic rather than establishing a general improvement in answer reliability. A larger independently designed evaluation could change both the size and direction of these differences.

The evidence supports a focused conclusion. Reference expansion recovered two missing clauses and helped the model include useful information in some reference questions. It also left retrieval failures unresolved, sometimes reduced expected answer content, and did not ensure supported or correctly cited answers. These findings explain the value of the implementation and identify the next steps more clearly than a single claim that GraphRAG is better.

VII. Conclusions and future work

This project began with a practical problem: answering a question about a regulation can require finding several connected passages, while a language model may give a plausible answer without finding or using all of them. The work addressed that problem by building a local assistant that combines an official source, text search, a reference graph and a language model. The resulting application lets users read the answer and inspect the evidence behind it.

The first contribution is the implementation of the complete process from the official publication to the user interface. The system transforms the source into traceable retrieval passages, extracts explicit internal references and preserves the source phrases behind those connections. It then searches this material, generates answers locally and exposes citations and evidence through a web interface. Docker Compose packages the supplied deployment, while saved records and command-line tools support inspection and reproduction. This makes the project an implemented system whose behaviour can be examined, rather than only a proposal for using a graph.

The second contribution is an evaluation that separates source recovery from answer quality. Text RAG and GraphRAG are compared under the same model, prompt and maximum evidence limits. Complete-clause coverage increases from 94.8% to 97.9%, with the gain concentrated in two reference questions. Their source traces show how following a reference can bring useful annex text into the model's input. These cases demonstrate a concrete benefit of reference-based graph expansion within the evaluated dataset.

The answer results are more mixed. Mean coverage of expected concepts rises from 86.7% to 88.3%, but improved retrieval does not always lead to a correct explanation. T22 shows a useful answer improvement, whereas T18 still assigns information to the wrong annex after the required clause has been retrieved. T31 shows that an answer can contain the expected words and contradict the source. These examples make clear that evidence selection, completeness, citation quality and support are different parts of the problem.

The conclusions that can be drawn from these results remain limited by the evaluation design. The questions were created and used during development, the two retrieval gains share an annex destination, and the study uses one small language model and a single generation experiment. The automatic content and support checks have known blind spots. The uncertainty intervals do not establish a general end-to-end advantage for GraphRAG. The appropriate conclusion is that explicit references can help recover connected evidence in some questions, while reliable use of that evidence remains unresolved.

The most valuable next step is an independent evaluation. Questions should be written by people who did not develop the system and should cover references to a wider range of articles and annexes. Expert review should assess whether each answer preserves the correct actor, condition and exception, and whether its citations support the claims. The existing passage links and saved outputs provide a useful starting point for that review.

Further experiments should examine retrieval and generation separately. A dense retriever and a neural reranker would provide stronger alternative baselines under the same evidence limit. A length-matched comparison would help distinguish the effect of graph selection from differences in actual context length. Larger local language models could test whether better generation makes more reliable use of the recovered passages. Each comparison should retain its inputs and outputs so that its results can be inspected.

The project ultimately shows both a useful mechanism and its boundary. Following a reference can help an assistant find a missing piece of the source. A reader still needs a clear answer, appropriate citations and access to the evidence to judge whether that piece was used correctly. Building those stages together, and examining them separately, is the main lesson of this work.

APPENDIX A
IMPLEMENTATION AND SCORING DETAILS

The main text explains the system through its purpose and execution flow. This appendix records the formulas and settings needed to inspect the implementation more closely. The details refer to the recorded comparison and its selected configuration. They are included so that another reader can connect the explanation to the code and saved evidence.

A. Source records and retrieval calculation

The reusable code is in the Python package `aiact_rag`. Within it, `corpus.py` prepares documents, structural units, passages and references; `retrieval.py` ranks and selects passages; `generation.py` builds requests and processes model responses; `verification.py` performs NLI checks; and `metrics.py` calculates evaluation measures. The browser connects through `web.py`, while the command-line interface calls the same core modules. Selected retrieval parameters are read from `data/retrieval_settings.json` by the CLI and API. Direct library calls need those settings passed explicitly if they are to reproduce the selected policy.

Documents, units, passages, references and retrieval edges are stored in JSONL files, a format with one JSON record per line. The RDF graph is also exported as Turtle. Source text remains separate from query-time scores and paths. For each passage c and its parent document d, the source-slice check requires

$$c.\text{text} = d.\text{text}[c.\text{start} : c.\text{end}],$$

using an inclusive start and exclusive end. Long structural units are split at up to 170 words. Overlap uses a short final sentence where possible, with a fallback of 20 words. Unit boundaries are preserved, and separately labelled lead-ins count towards the evidence budget.

BM25 uses lowercased English terms supplied by CountVectorizer, with stop-word removal. For passage c and question q, the lexical score is

$$L(c,q) = \sum_{t \in q} \log\left(1 + \frac{N - n_t + 0.5}{n_t + 0.5}\right) \frac{f(t,c)(k_1+1)}{f(t,c) + k_1(1 - b + b|c|/\overline{|c|})}.$$

Here, N is the number of passages, n_t is the number containing term t, $f(t,c)$ is the term count in the passage, and $|c|$ is passage length in indexed terms. The average indexed length is $\overline{|c|}$. The implementation uses $k_1 = 1.5$ and $b = 0.75$, with binary query-term presence. Dividing by the maximum lexical score for that question gives the normalised score $\ell(c,q)$; a zero maximum gives zero normalised scores.

Shared adjustments produce the base score used by both retrieval modes. An explicitly named document receives a 0.35 boost, and a matching article paragraph receives a further 0.15. Recital scores are multiplied by 0.85 before the paragraph adjustment. These are ranking preferences, not measures of the legal authority of a passage. The TF-IDF ablation uses unigrams and bigrams over the same passage strings.

The graph query asks for document neighbours. A simplified outgoing query, with namespace prefixes assumed to be defined, is:

```
SELECT DISTINCT ?seed ?neighbour WHERE {
  VALUES ?seed { act:art_23 }
  ?seed v:mentions ?neighbour .
}
```

The actual query operates on validated corpus identifiers; raw user text is not inserted as an unrestricted SPARQL fragment. Concrete occurrence records are checked against the returned neighbours. A route retains its source and target, occurrence identifier, phrase, positions, units, direction and hop count.

Let B_s be the largest base score in seed document s. For a reference e, let r_e be the largest normalised lexical score in the source unit containing that reference. A candidate passage receives a bonus

$$g(c,q) = \max_{\pi \rightsquigarrow c} \left[\alpha B_s \prod_{e \in \pi} (0.6 + 0.4 r_e)\, 0.7^{h-1} \tau_c \right],$$

where h is the route length, α is the selected graph weight and τ_c is 1.15 for a resolved target unit, or 1 otherwise. The final score adds this bonus to the adjusted base score. For each reference, at most the two highest-base-score passages in the target unit are considered; without a unit target, the two best passages from the destination document are used.

The selected values are $\alpha = 0.25$, two outgoing hops and at most two seeds. Expansion allows at most six next-document candidates per origin and twelve frontier records per depth. Cycles within a route are excluded. Selection retains at most six passages and three per document, with a total of at most 850 words including lead-ins. Windows overlapping more than 40% of the shorter window within the same unit are suppressed. Identifier order breaks score ties deterministically.

Development selection tests $\alpha \in \{0, 0.25, 0.5, 0.8, 1.2\}$, one or two hops, and outgoing or bidirectional traversal, with one disabled condition at zero. The resulting 17 configurations are ordered first by mean complete-clause coverage, then by key-quote coverage, source-document precision, lower graph weight and fewer hops. The random control swaps directed document pairs with a fixed seed, preserves each document's incoming and outgoing degree, avoids self-links and duplicate edges, and removes precise target-unit information.

For T22, the selected seed is Article 86. The graph contribution to the relevant annex passage is

$$0.25 \times 1.35 \times (0.6 + 0.4 \times 0.803466) \simeq 0.310968.$$

Adding this to its base score of 0.324983 gives 0.635951. The route is the recorded occurrence `edge_00393`, whose source phrase is "Annex III". The selected GraphRAG passage order is `art_86_c002`, `art_86_c001`, `art_86_c000`, `anx_III_c001`, `anx_III_c003` and `art_3_c061`. This record supports the worked example without interpreting the scores as probabilities.

B. Generation, measurement and saved provenance

The generator uses Qwen2.5-1.5B-Instruct Q4_K_M GGUF with llama.cpp b10809, CPU execution and a 4,096-token context. The request specifies temperature 0, seed 17, at most 320 output tokens and `cache_prompt:false`. Server defaults in the runtime manifest are distinct from these explicit request settings. The pinned model file is checked by SHA-256 before scored execution, and the manifest retains the runtime identity and effective request options.

Citation parsing recognises grouped identifiers separated by commas, semicolons or spaces, and counts malformed source-like labels as invalid. Citation blocks following a full stop attach to the preceding sentence. When several citation blocks appear together, all of them are associated with that sentence for support checking.

The NLI checker uses the pinned `cross-encoder/nli-MiniLM2-L6-H768` model with quantised ONNX execution. Premises use overlapping windows of at most 180 words and a 512-token model limit. A sentence is counted as supported when its maximum entailment score reaches 0.8. Reference support uses annotated clauses; cited-sentence support uses actual cited passages. The implementation name `citation_support_precision` denotes the latter conditional sentence measure, not citation-level precision.

Document recall measures the fraction of required documents retrieved. Complete-clause recall tests whether the union of selected source intervals covers each annotated interval in full. Source-document precision measures the fraction of selected passages whose parent document is annotated as required. A passage can therefore count favourably under document precision while missing the required clause. Key-quote recall checks short distinctive source spans separately.

Answer facets use predefined regular expressions, with numerical values and units bounded to prevent accidental partial matches. Citation text is removed before matching. Full refusals and partial uncertainty are distinguished; a partial uncertainty statement does not erase another supported answer part. Each measure is calculated per question or answer before averaging. Undefined scores are excluded from that measure's denominator. This is why the citation-support sample sizes differ from the answerable-question total.

The run uses shuffled questions with seed 173 and a cyclic rotation of modes. Identical complete requests share a marked recorded response, so there are 107 actual scored calls and 13 reuses across 120 records. The repeat probe contains two additional development calls. Paired bootstrap summaries use 5,000 resamples with a fixed seed. They retain the pairing of systems within each question.

The recorded results are stored under `results/v3_1_2026-09-09`. The raw file `generations.jsonl` contains generated answers and request information; `scored_generations.jsonl` adds the scores. Separate files hold retrieval, development selection, ablations, summaries and the repeat probe. `answer_audit.json` exposes individual cases, while the review forms remain unfilled.

The experiment retains source, benchmark, code, model and library fingerprints. These records connect each saved score to its inputs and runtime settings. A consistency check requires those inputs and settings to match when resuming an experiment. Each experiment has its own identifier.

The web API provides endpoints for system status, retrieval, answer generation, graph inspection and saved evaluation results. Requests specify a question and one of three modes: `llm`, `text` or `graph`. The browser answer workflow runs the NLI check when retrieved evidence is available, while the basic CLI `ask` command validates citation identifiers without automatically running NLI. The experiment scorer performs its own NLI assessment using the reference annotations. These assessment paths differ, but retrieval and generation continue to use the same core implementation.

APPENDIX B
INSTALLING, RUNNING AND INSPECTING THE PROJECT

This appendix explains how to use the supplied application and examine its evidence. It separates starting the service from running a new experiment. The existing results can be inspected without generating new model answers. The accompanying repository is available at https://github.com/rubengamarro/aiact-assistant.

A. Starting the local application

The supplied deployment requires Docker with Compose v2. On Windows and macOS, Docker Desktop must use Linux containers. The images target Linux/amd64 and CPU inference; ARM emulation has not been validated. No host Python installation or GPU is required. Initial setup needs internet access to download software and model assets.

Open a terminal in the code repository and run:

```
docker compose up --build
```

The first startup can take several minutes. The application image includes the saved official HTML and processed corpus. During the build, `scripts/verify_corpus.py` reconstructs the corpus in a temporary location and compares it with the packaged files. Initialisation downloads and verifies the model assets and creates the API key. The model service starts after initialisation succeeds, and the application waits for model health.

When startup completes, open http://127.0.0.1:8000. Enter an English question and select a mode. "Inspect retrieval only" shows the evidence without asking the model to produce an answer. "Generate answer" adds model generation and the available automatic support checks. Clicking a recognised citation opens the corresponding retrieved passage. To exercise the same core functions from the terminal, use:

```
docker compose exec app python -m aiact_rag retrieve "Article 49 national
    registration" --mode graph
docker compose exec app python -m aiact_rag ask "What must provider-controlled
    logs retain?" --mode text
```

The supplied test suite and corpus check can be run while the application is available:

```
docker compose exec app python -m pytest -q --disable-warnings
docker compose exec app python scripts/verify_corpus.py
```

These commands verify software behaviour and source consistency. The checks cover the installed application and its corpus; they do not rate the correctness of all possible answers.

If the application does not become healthy, inspect the startup logs:

```
docker compose logs --tail=100 init model app
```

The logs help distinguish an asset-download problem, a model-startup problem and an application error. The optional `.env` file can be copied from `.env.example` to change `WEB_PORT` or `LLAMA_THREADS`. Their defaults are 8000 and 6. If the port changes, use that value in the browser address. Changing the thread count also changes the runtime conditions of a new experiment.

Stop the application with:

```
docker compose down
```

This stops and removes the containers while keeping their persistent volumes. Models, runtime credentials and container-side results remain available at the next startup. Adding `--volumes` also deletes that stored material. Repeating the startup command restarts the application or rebuilds it after code changes.

B. Inspecting the recorded experiment or running a new one

The published comparison is identified by `v3_1_2026-09-09`. Its report and saved answers are included in the repository. The report explains the metrics, and the raw and scored records allow a reader to follow an aggregate value back to individual questions. To audit the existing records without generating new answers, run:

```
docker compose exec app python scripts/audit_run.py --run-id v3_1_2026-09-09
```

The audit checks that the saved answers, metrics and source records are consistent with the experiment's manifests. It does not generate model answers.

For a new experiment, choose an unused identifier and run the following commands from the repository while the application is healthy:

```
docker compose exec app python scripts/build_benchmark.py
docker compose exec app python scripts/evaluate.py --run-id my_reproduction
docker compose exec app python scripts/audit_run.py --run-id my_reproduction
docker compose exec app python scripts/make_report.py --run-id my_reproduction
docker compose cp app:/app/results/my_reproduction ./results/my_reproduction
```

The benchmark builder checks source anchors. Evaluation performs development selection, retrieval, generation and scoring according to the protocol. The audit checks the saved records, and the report script produces the readable summary and answer audit. The last command copies the new experiment from the container's results volume to the host repository. The host's `results` directory is not automatically synchronised with that volume.

Use a distinct identifier for each experiment. The `--prepare-only` option performs development selection and test retrieval without completing generation. The `--score-only` option computes metrics and NLI from saved responses under the recorded configuration. Resuming generation requires matching source data, benchmark, code and runtime settings.

Reproduction should retain the exact original HTML, model and executable hashes, selected settings, prompts and raw outputs. A source identifier is meaningful only in relation to the source and parser that created it. Model outputs may vary across machines, so the runtime settings and complete responses are stored with each experiment.

REFERENCES

[1] European Parliament and Council of the European Union, "Regulation (eu) 2024/1689 laying down harmonised rules on artificial intelligence," Official Journal of the European Union, 12 July, 2024, original English publication. [Online]. Available: https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng

[2] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474. [Online]. Available: https://arxiv.org/abs/2005.11401

[3] C. D. Manning, P. Raghavan, and H. Schuetze, *Introduction to Information Retrieval*. Cambridge University Press, 2008. [Online]. Available: https://nlp.stanford.edu/IR-book/html/htmledition/okapi-bm25-a-non-binary-model-1.html

[4] R. Cyganiak, D. Wood, and M. Lanthaler, "Rdf 1.1 concepts and abstract syntax," W3C, Tech. Rep., 2014. [Online]. Available: https://www.w3.org/TR/rdf11-concepts/

[5] S. Harris and A. Seaborne, "Sparql 1.1 query language," W3C, Tech. Rep., 2013. [Online]. Available: https://www.w3.org/TR/sparql11-query/

[6] Z. Shen, C. Diao, P. Vougiouklis *et al.*, "GeAR: Graph-enhanced agent for retrieval-augmented generation," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 12 049–12 072. [Online]. Available: https://aclanthology.org/2025.findings-acl.624/

[7] D. Edge *et al.*, "From local to global: A graph rag approach to query-focused summarization," 2024. [Online]. Available: https://arxiv.org/abs/2404.16130

[8] T. Gao, H. Yen, J. Yu, and D. Chen, "Enabling large language models to generate text with citations," in *Proceedings of EMNLP*, 2023. [Online]. Available: https://aclanthology.org/2023.emnlp-main.398/

[9] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated evaluation of retrieval augmented generation," in *Proceedings of EACL: System Demonstrations*, 2024. [Online]. Available: https://aclanthology.org/2024.eacl-demo.16/

[10] Sentence Transformers contributors, "Cross-encoder for natural language inference: nli-minilm2-l6-h768," 2026, revision b95119ce93d3e065de6214e38cd4a97b0f2f2c6d; quantised ONNX; SNLI and MultiNLI. [Online]. Available: https://huggingface.co/cross-encoder/nli-MiniLM2-L6-H768

[11] RDFLib contributors, "Rdflib documentation," 2025. [Online]. Available: https://rdflib.readthedocs.io/

[12] Qwen Team, "Qwen2.5-1.5B-Instruct-GGUF: Model card and quantised weights," 2024, revision 91cad51170dc346986eccefdc2dd33a9da36ead9; Q4_K_M. [Online]. Available: https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct-GGUF

[13] ggml-org contributors, "llama.cpp: Local inference and http server," 2026. [Online]. Available: https://github.com/ggml-org/llama.cpp/blob/b10809/tools/server/README.md